

Community Emergency Response App (CERA)

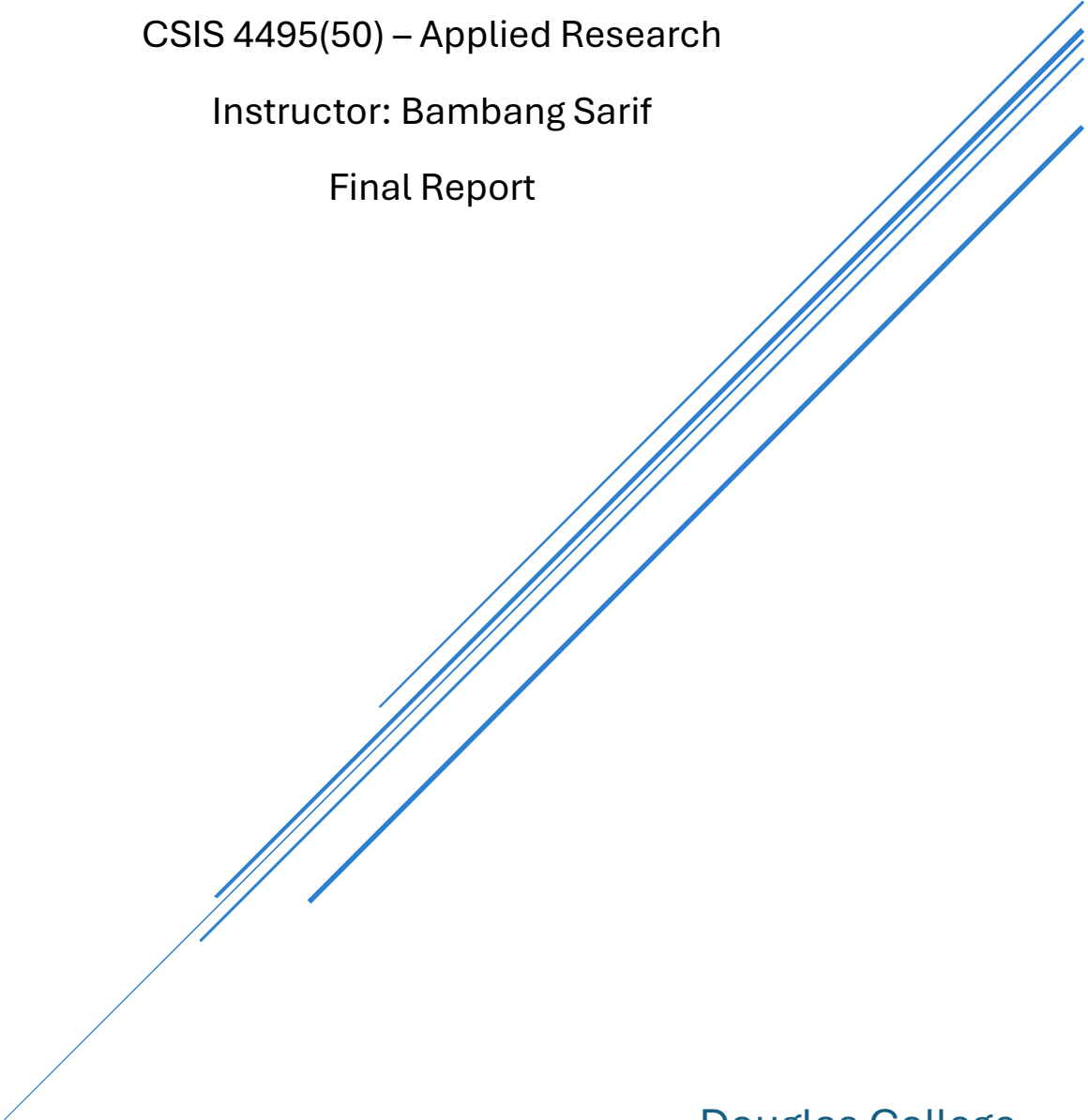
Name: Dilpreet Sandhu(300389106)

Harleen Kaur(300389855) (Leader)

CSIS 4495(50) – Applied Research

Instructor: Bambang Sarif

Final Report

A series of five parallel blue lines of varying lengths, slanted diagonally from the bottom-left towards the top-right, located in the lower right quadrant of the page.

Douglas College
CSIS 4495

COMMUNITY EMERGENCY RESPONSE APP (CERA) – PROJECT PROPOSAL

Table of Contents

1. Introduction.....	2
2. Proposed Research Project	
2.1 Problem Definition.....	2
2.2 Research Questions.....	3
2.3 Proposed Solution.....	4
2.4 Expected Outcomes.....	5
3. Project Planning And Timeline	
3.1 Overview.....	6
3.2 Week-by-Week Breakdown.....	7
3.3 Summary of Deliverables.....	8
4. Implemented Features	
4.1 Feature A- Resident: Incident Reporting.....	9
4.2 Feature B- Coordinator: Incident Management & Volunteer Dispatch.....	12
4.3 Feature C - Volunteer: Task Acceptance & Status Updates	18
4.4 Feature D - Coordinator: Incident Tracking	23
5. Reflection.....	26
6. Concluding Remark.....	28
7. Appendix A - Installation Guide.....	28
8. Appendix B – User Guide.....	31
9. Appendix C- Dataset and API Used.....	31
10. Conclusion.....	36

INTRODUCTION

Emergencies such as fires, medical incidents, natural disasters, and unexpected accidents can occur at any time, often leaving individuals unsure of how to respond or whom to contact. In these moments, timely communication is crucial. The faster accurate information reaches the right people, the greater the chances of minimizing harm and coordinating an effective response. Yet many communities still lack a unified system that supports quick reporting and organized volunteer involvement.

In most cases, people depend on informal methods such as phone calls, text messages, or social media posts to share urgent information. While widely used, these channels are not designed for emergency response—they may delay communication, fail to reach the necessary individuals, or provide incomplete details. Without a structured process, incidents can escalate due to slow response times, miscommunication, or uneven distribution of help.

To address these challenges, this project introduces the Community Emergency Response App (CERA)—a mobile-first platform designed to streamline emergency reporting, volunteer coordination, and communication between residents, volunteers, and community coordinators. CERA aims to provide a reliable, accessible, and easy-to-use system that helps communities respond more effectively by reducing confusion and improving the flow of information during urgent situations.

This report outlines the background of the problem, the research and design considerations that shaped the proposed solution, and the full development timeline of the application. Additionally, it summarizes the outcomes of the implementation and reflects on key insights gained throughout the project lifecycle.

PROPOSED RESEARCH PROJECT

This section outlines the approved research direction for the Community Emergency Response App (CERA), including the problem being addressed, guiding questions, proposed solution, and expected outcomes. The project combines applied research with practical mobile application development to create a community-focused emergency coordination system.

2.1 Problem Definition

Community-level emergency response faces significant challenges. When an incident occurs, residents often depend on informal methods—phone calls, group chats, or social media—to communicate what is happening. While easy to use, these methods are

unreliable and lack structure. Messages may be missed, shared too late, or arrive without accurate location details, which delays the ability of volunteers or coordinators to respond.

Through our research, four core problems were identified:

1. No standardized tool for quick, accurate emergency reporting

Residents lack a simple, reliable way to submit urgent reports with essential details like photos, descriptions, and geolocation. This leads to delays and inconsistent information.

2. Volunteers remain unaware of emergencies happening nearby

Even when willing and available, volunteers often have no system to notify them of incidents in their area. As a result, communities lose valuable response time that trained volunteers could have provided.

3. Coordinators lack oversight and structured workflows

Coordinators need a clear system to view, verify, assign, and track emergencies. Without it, they rely on manual communication, which is slow and prone to errors.

4. Communication between all three groups is fragmented

Residents, volunteers, and coordinators operate in isolated channels, leading to confusion, duplicated efforts, and lack of follow-up.

These problems highlight the gap that CERA aims to fill: creating a unified digital platform that strengthens communication and coordination during emergencies.

2.2 Research Questions

To guide the design and development of CERA, this project focuses on three main research questions:

1. How can a mobile application enable residents to report emergencies quickly and reliably?

This question focuses on designing a reporting process that minimizes the time and cognitive effort required during stressful situations. The research examines interface simplicity, clear input fields, automated GPS location capture, and support for photo evidence. The intention is to determine how the reporting workflow can be streamlined so residents are able to submit complete and accurate emergency reports in under a minute, even if they are panicked or unfamiliar with technology.

2. **What is the most effective way to connect nearby volunteers with active incidents?**

This question explores the logic behind matching volunteers to incidents based on proximity and availability. It investigates how real-time location data, volunteer status updates, and geospatial filtering can be used to identify the closest suitable volunteers. The study also considers notification delivery timing and the decision-making process for volunteers when accepting or declining tasks. The goal is to design a system that significantly reduces response delays by reaching the right volunteers immediately.\

3. **Can real-time communication and location tracking reduce overall emergency response time?**

This question examines whether continuous updates improve coordination among residents, volunteers, and coordinators. It focuses on the role of live incident status changes, map-based visualization, and two-way communication during an emergency event. The research evaluates how real-time information sharing can reduce confusion, prevent duplicate efforts, and support a more organized emergency response process.

2.3 Proposed Solution

The proposed solution for this project is a mobile-first platform that connects three groups—Residents, Volunteers, and Coordinators—into one unified emergency response system. Each role is designed to address a specific need, but the real value comes from how these roles interact with each other in a structured and transparent workflow.

Residents

For residents, CERA provides a straightforward and reliable way to report emergencies. The reporting interface is designed with simplicity in mind: residents can capture a photo, write a short description, and rely on automatic GPS detection for precise location data. This reduces the number of steps required and ensures that essential details are captured even in high-stress situations.

In addition to emergency reporting, residents have access to an **Emergency Contacts module**. This feature allows users to store personal emergency numbers—such as family members or trusted contacts—directly within the app. Residents can quickly call these contacts during urgent situations or share their live location with them. Including this feature strengthens the app’s value by addressing not just community-level emergencies but also personal safety needs.

Volunteers

Volunteers form the operational backbone of the response flow. When an incident is reported, the system identifies volunteers who are nearby and currently available, and sends them timely notifications. Volunteers can accept or decline an assignment with a single tap, ensuring only willing and available individuals are dispatched.

They can also manually update their location, allowing the coordinator's dashboard to accurately reflect their proximity to active incidents. Throughout the response process, volunteers maintain communication with coordinators and update task statuses as they progress.

Coordinators

Coordinators manage and oversee the entire emergency landscape within the app. They review newly submitted incident reports, verify them, and assign appropriate volunteers. A centralized dashboard gives coordinators a real-time view of active incidents, volunteer activity, and response progress.

Additionally, coordinators handle volunteer approval, ensuring that only verified users join the response network. This step improves safety, quality control, and trust within the community.

Integrated Workflow

By integrating these three roles into one cohesive system, CERA eliminates the fragmentation that currently slows community-level response efforts. Instead of relying on scattered communication channels, the platform creates a continuous pipeline—from reporting to dispatching to resolution—supported by real-time data and clear role-based responsibilities. This ensures faster response times, improved coordination, and a safer, more organized emergency management process for the community.

Note: Features such as offline reporting and offline access, although included in the initial proposal, were not implemented in the final prototype due to time constraints.

2.4 Expected Outcomes

The development of CERA is expected to produce both a working technical prototype and meaningful insights into how technology can support community-driven emergency response. The primary outcome of this project is a complete mobile application that demonstrates core features such as emergency reporting, volunteer dispatch, real-time status updates, manual location sharing, and push notifications. These components together form the foundation of a functional emergency coordination system that residents, volunteers, and coordinators can use in simulated scenarios.

Another major outcome is improved communication during emergencies. By replacing scattered tools—such as phone calls, text messages, and informal social media posts—

with a single unified platform, communities can experience faster, clearer, and more structured communication. This contributes to better coordination and more consistent response flows.

The project is also expected to establish a standardized model for managing community-level emergencies. Through clearly defined roles, structured workflows, and integrated communication channels, CERA provides a reliable process for reporting, dispatching, and overseeing emergencies. This framework can serve as a reference for similar systems.

Throughout development, the project will also uncover a range of technical and practical challenges. Issues such as the accuracy of geolocation data, the speed of notification delivery, the engagement level of volunteers, limited real-world testing opportunities, and the impact of network connectivity will provide valuable lessons. These insights will inform recommendations for improvement and highlight areas where further research or optimization is needed.

Finally, the project aims to create a foundation for potential real-world adoption. If validated through testing, the system could be extended or modified for use by municipal emergency departments, community volunteer groups, or disaster-relief organizations. The knowledge gained through this project—both technical and experiential—will support future enhancements, scaling decisions, and integration opportunities.

PROJECT PLANNING AND TIMELINE

The development of CERA followed an iterative, milestone-based timeline that moved from planning and research into hands-on implementation, testing, and refinement. Although the project began with a structured Gantt chart, the timeline evolved as technical challenges emerged, and features were refined based on testing and feasibility.

3.1 Overview of the Development Timeline

The project was planned across fifteen weeks and divided into logical phases that supported continuous progress. Early weeks focused on research, requirement gathering, and interface planning. Mid-project weeks transitioned into full development of core features such as authentication, incident reporting, volunteer dispatch, and geolocation. The final weeks focused on debugging, polishing the user interface, writing technical documentation, and preparing for the final defense.

3.2 Week-by-Week Breakdown

Weeks 1–3: Project Preparation and Proposal

The first phase involved selecting the project topic, conducting background research, and preparing the formal proposal. During this period, the team explored existing emergency and volunteer coordination apps, identified gaps in current solutions, and finalized the concept of CERA. A shared GitHub repository was created, wireframes were drafted, and responsibilities were divided between team members.

Weeks 4–5: Initial App Setup and Backend Structure

The team set up the React Native mobile project and implemented the basic navigation flow. In parallel, the Express.js backend was created with initial folder structure, user model, and routing setup. Both frontend and backend environments were configured to communicate with each other, creating the foundation for further development.

Weeks 6–7: Authentication and Role-Based Access

During this period, user registration, login, and JWT-based authentication were implemented. The system was expanded to correctly support three roles—Resident, Volunteer, and Coordinator. Coordinators were given access to approve or decline volunteer applications. Password hashing, input validation, and secure API access were also completed.

Weeks 8–9: Incident Reporting and Volunteer Dispatch

This milestone marked a major shift toward core application functionality. The team implemented the Resident’s incident reporting flow, including image upload, description fields, severity levels, affected count, and automatic GPS location detection. The backend was updated to store incidents and handle dispatch logic. Volunteers gained the ability to receive assignments, accept or decline tasks, and track status changes.

Weeks 10–11: Maps, Geolocation, and Nearby Volunteer Logic

The project incorporated geolocation and distance-based volunteer filtering using MongoDB’s geospatial features. A manual location update feature was added for volunteers, ensuring reliable proximity matching. This period also included improvements to location APIs and refining how coordinators view volunteers and incidents based on distance.

Weeks 12–13: Push Notifications and Supporting Features

Push notifications were integrated using Expo, enabling coordinators to notify volunteers instantly when they are assigned to an incident. Additional supporting features such as Emergency Contacts for residents were implemented. Although offline mode was included in the proposal, it was not implemented due to time limitations and technical prioritization.

Week 14: Testing, Debugging, and Midterm Preparation

The team performed extensive testing on all primary features, including incident creation, dispatch assignments, user role flows, and notification handling. UI improvements and bug fixes were completed. A midterm demo video was created, demonstrating the app's core functionality in action.

Week 15: Final UI Polish, Documentation, and Defense Preparation

During the final week, the user interface was refined to ensure consistency, clarity, and accessibility. The team completed the final documentation—including design descriptions, implementation details, and reflections—and prepared for the final defense presentation. GitHub repositories were reviewed and updated to ensure all code and resources were properly organized.

3.3 Summary of Deliverables

Throughout the timeline, the team successfully delivered:

- A fully functional cross-platform mobile prototype
- A Node.js backend with real-time dispatch logic
- A structured workflow for reporting, dispatching, and tracking incidents
- Role-based user access and volunteer approval
- Push notifications and geolocation features
- Emergency contact management for residents
- Technical documentation and final presentation materials

IMPLEMENTED FEATURES

This section provides detailed documentation of the major features implemented in the Community Emergency Response App (CERA). Each feature description includes an introduction, purpose, assumptions, challenges, changes made during development, and technical implementation details. Screenshots with figure titles are also provided to illustrate the functionality. The features are presented in separate subsections for clarity.

4.1 Feature A — Resident: Incident Reporting

Introduction

The Incident Reporting feature enables **both residents and volunteers** to quickly and accurately report emergencies from their mobile devices. This feature represents the starting point of the entire emergency-response workflow in CERA. It was designed to be simple, fast, and reliable, allowing users to submit complete emergency information even in stressful situations. The interface supports photo evidence, automatic GPS capture, text descriptions, incident categorization, and severity levels, ensuring that reports contain the essential details required for coordinators and volunteers to respond effectively.

Feature Description

The reporting screen allows users to submit a new incident by selecting an incident type, writing a description, choosing a severity level, and optionally entering the estimated number of people affected. Integrated camera functionality allows users to capture a photo of the situation, helping coordinators verify the nature and urgency of the incident. To ensure precise location data, the system retrieves the user's current GPS coordinates using the Expo Location API and converts them into a readable address whenever possible.

All collected information is combined and submitted to the backend as a structured report. After submission, the user receives an in-app confirmation through toast notifications, and the system redirects them to the home screen, ensuring a clear and smooth reporting experience.

Assumptions Made During Implementation

The design assumes that users will have access to a working internet connection and active GPS services when submitting a report. It also assumes that mobile devices support camera permissions and that users prefer a guided structured form instead of

large free-text fields. These assumptions shaped how data is collected and validated before submission.

Changes Made During Development

Several adjustments were made during development to improve usability and clarity. A dedicated “**Use My Location**” button was added to let users manually confirm their location instead of relying solely on automatic GPS pulls. Validation messages were improved to guide users when required fields were missing. The interface layout was refined to create a cleaner flow between photo preview, location details, and severity options. These changes strengthened overall usability and reduced user confusion.

Technical Implementation Details

The incident reporting module was implemented using React Native and Expo, acting as the main interface for data submission. Location detection is handled through the Expo Location API, which retrieves GPS coordinates and attempts reverse-geocoding to obtain a readable address. Photo capture is supported through Expo ImagePicker, which returns the image in Base64 format along with its dimensions to ensure accurate previews.

All the collected data—including incident type, custom type (if provided), description, severity, affected count, and the GeoJSON-formatted location object—is packaged using FormData. The photo is uploaded as a multipart form file. This data is then securely sent to the backend using Axios.

On the backend, the Express.js server receives the submission through the /incidents endpoint. The server processes the uploaded image, extracts the form fields, and creates a new MongoDB record. Each incident document stores the reporter’s user ID, textual details, severity level, number of affected people, photo reference, timestamps, and location stored as a **GeoJSON Point** to support geospatial queries. Every new incident is automatically set to **pending**, allowing coordinators to review and verify it before dispatching volunteers.

Screenshots

The interface includes interactive incident-type buttons, a multiline description box, severity options, a “Capture Photo” button, and a “Use My Location” button. After capturing, the photo appears as a preview with an option to retake it. Location coordinates or the readable address appear once retrieved, providing visual confirmation before submission

Community Emergency Response App

10:55



CERA

←

Report an Incident

Incident Type *

Fire

Medical Emergency

Flood

Earthquake

Accident

Crime

Others

Description *

Describe the situation briefly...

Severity Level *

Low

Medium

High

Estimated People Affected

e.g., 5

Capture Photo

Use My Location

Submit Report

Your report will be sent to nearby volunteers and coordinators.

 Home

 Report

 Incidents

 Profile

10:55



CERA

←

Report an Incident

Incident Type *

Fire

Medical Emergency

Flood

Earthquake

Accident

Crime

Others

Description *

Fire in townhouse

Severity Level *

Low

Medium

High

Estimated People Affected

7

Capture Photo

Use My Location

Submit Report

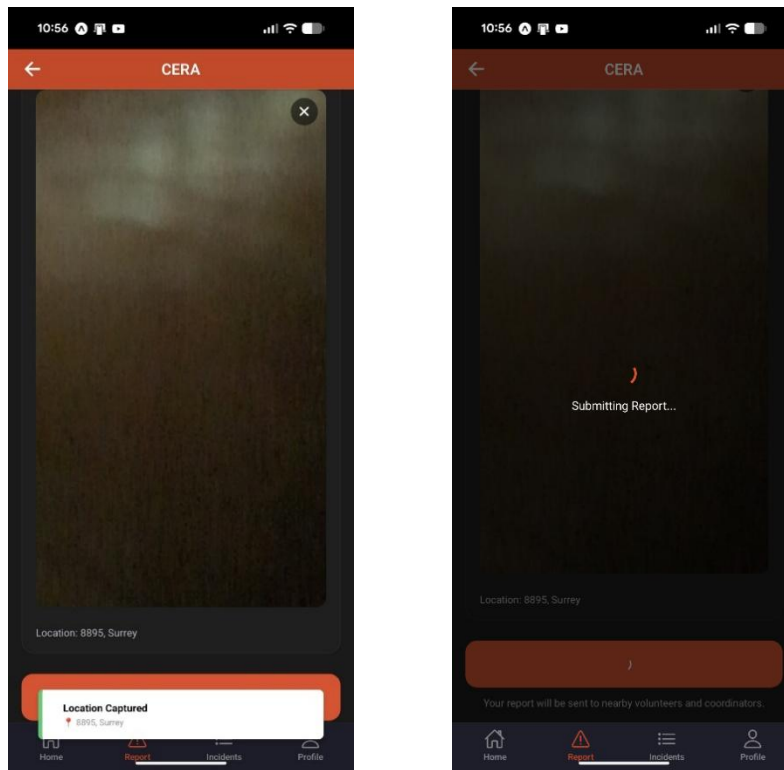
Your report will be sent to nearby volunteers and coordinators.

Home

Report

Incidents

Profile



Conclusion

The incident reporting feature provides a fast, structured, and user-friendly method for documenting emergencies within the CERA system. By combining geolocation, multimedia evidence, form validation, and backend integration, it ensures that incidents are recorded clearly and accurately. As the entry point for the entire emergency-response workflow, this feature plays a critical role in enabling coordinators and volunteers to take timely and informed action.

4.2 Feature B — Coordinator: Incident Management & Volunteer Dispatch

Introduction

The Coordinator Dashboard is the core management feature of CERA, providing coordinators with full oversight of all reported incidents. This module enables coordinators to review new reports, approve them, assign volunteers, monitor progress, and view incidents across multiple statuses. It is designed to support fast decision-making, clear visibility, and an organized workflow from report submission to resolution.

Feature Description

The dashboard displays a complete list of incidents categorized by status, including Pending, Approved, Assigned, In Progress, Resolved, and Completed. Coordinators can switch between these categories using a horizontal filter bar. This filtering system allows coordinators to quickly navigate large volumes of reports and focus only on incidents requiring action.

Each incident is rendered using the IncidentCard component, which displays key details such as type, severity, photo, description, and timestamp. For pending incidents, coordinators can approve them directly from the card, which updates the backend and refreshes the list.

Once an incident is approved, the coordinator can dispatch volunteers by opening a dedicated assignment modal. The system automatically fetches only those volunteers who are approved, certified, and currently marked as available, ensuring that the dispatch process is efficient and accurate. The coordinator can select one or multiple volunteers, and the system sends the assignment to the backend.

Assumptions Made During Implementation

The feature was developed with the assumption that coordinators must handle incidents at different stages simultaneously. It was also assumed that coordinators require visibility across all incidents, including previously completed ones, to maintain proper records. The design assumes that the volunteer pool may vary in availability, certification, or approval status, which is why the backend filters were applied.

The feature also assumes that coordinators will manually confirm dispatch decisions rather than relying on automated assignment algorithms. This keeps decision-making under coordinator control, which is suitable for community-based emergency response.

Changes Made During Development

A few refinements were introduced during the development of this feature to make the dashboard more intuitive, accurate, and reliable.

First, the filtering system was enhanced to ensure a clearer workflow. Completed incidents were included under the “Resolved” filter so that coordinators could view all finalized cases without switching between multiple categories.

Another major refinement was added to the dispatch process. Only volunteers who are approved, certified, and currently marked as available are shown in the dispatch modal. This prevents coordinators from accidentally assigning incidents to volunteers who are offline or unavailable.

A further improvement was made to support real-world volunteer responses. When a volunteer declines an assigned task, the incident automatically moves back to the Approved section so the coordinator can dispatch another volunteer without searching for the report manually. This ensures that no incident remains unattended, even when volunteers reject the assignment.

Together, these changes improved the accuracy of the dispatch flow and ensured that coordinators maintain full control over the emergency management process.

Technical Implementation Details

This feature is implemented using React Native with a combination of state management, backend API integration, and custom UI components. When the screen loads, it performs multiple API calls to retrieve incidents across all statuses. These responses are merged into a single list and stored in state.

Filtering is handled by transforming the selected filter into a normalized format and comparing it against the status field of each incident. Special handling was implemented for “Resolved,” which includes both "resolved" and "completed" statuses.

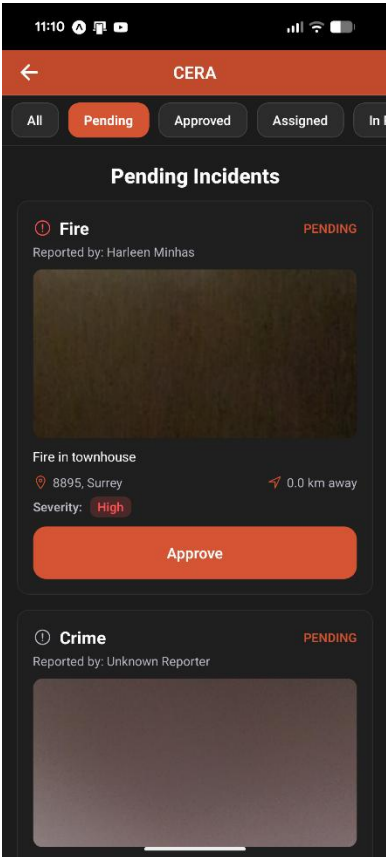
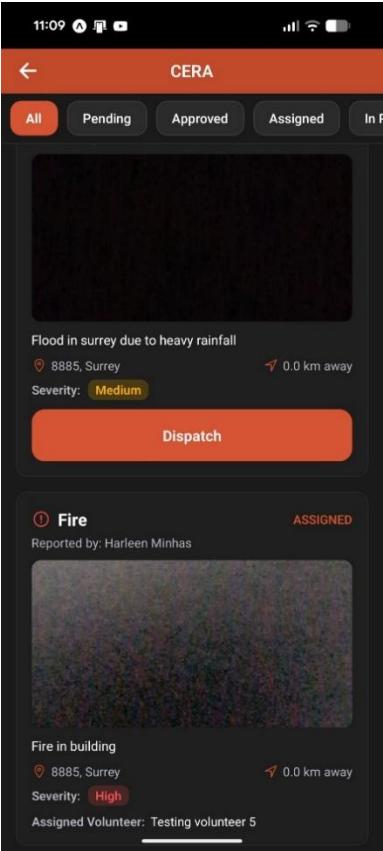
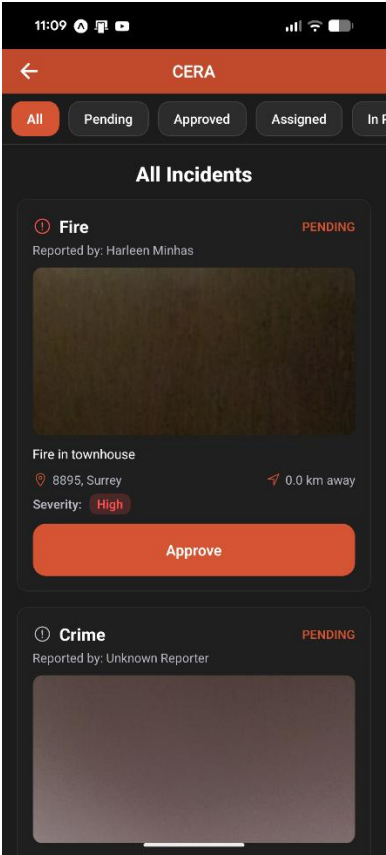
Volunteer dispatch is managed through a modal window. When the coordinator opens the dispatch modal, the system calls the backend with query parameters (role=volunteer, approved=true, certified=true, available=true) to ensure only valid volunteers are shown. The coordinator can toggle volunteer selections, and the final list is submitted to the backend using the /incidents/:id/dispatch route.

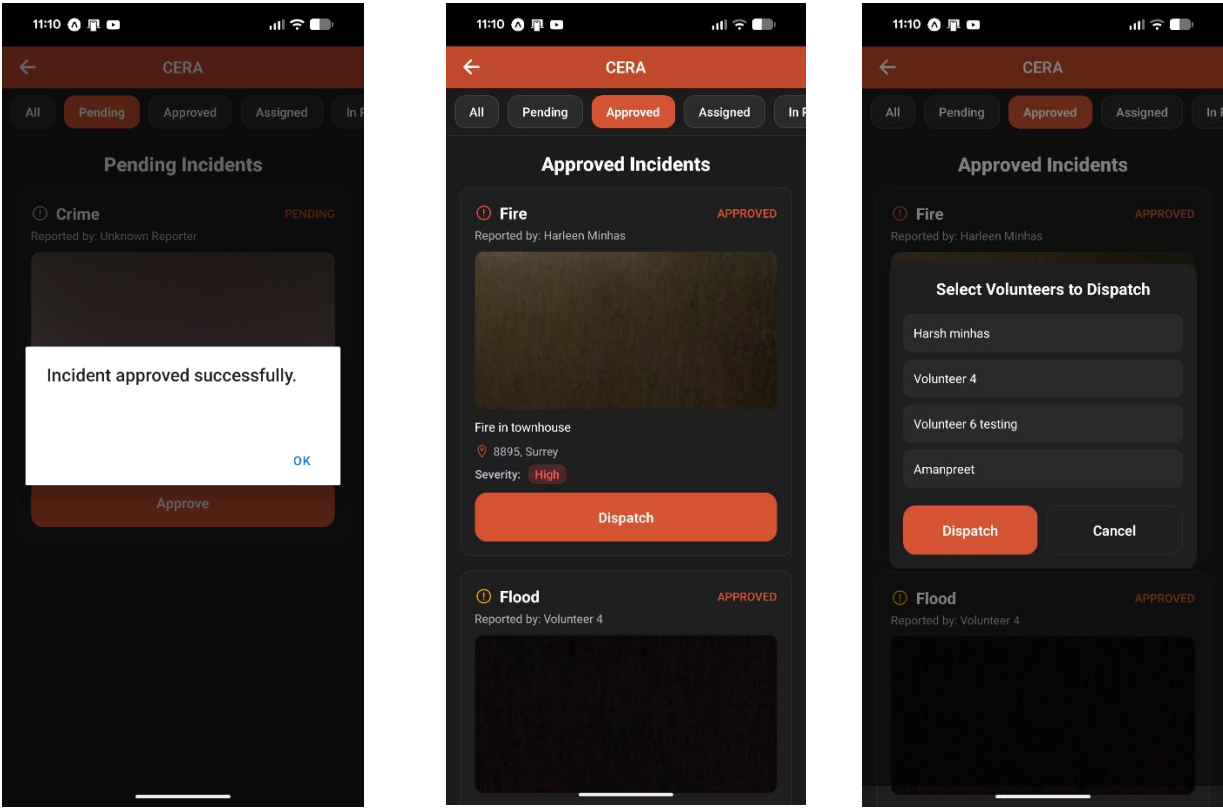
The API integration is handled through Axios, with appropriate error handling for approval failures, network issues, or volunteer assignment problems. The UI components—including filter buttons, modal, and volunteer list—are styled using dynamic theme colors to match light and dark modes.

Screenshots

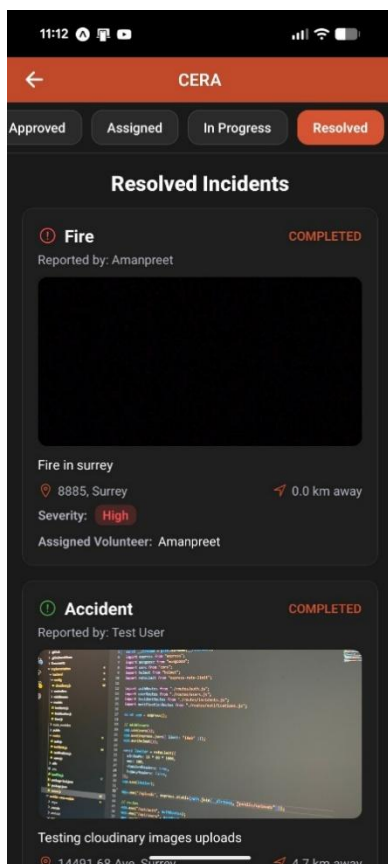
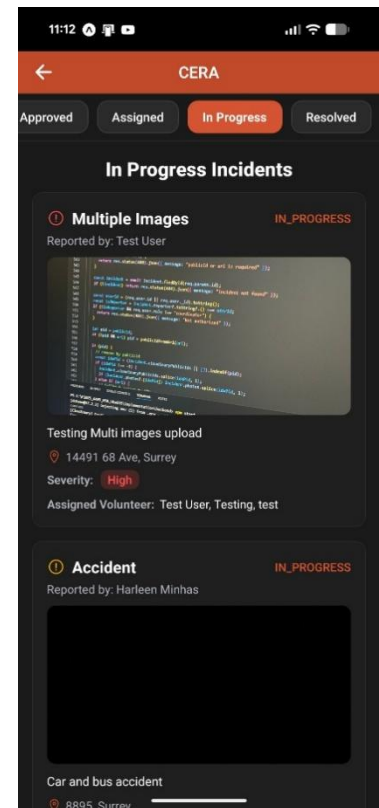
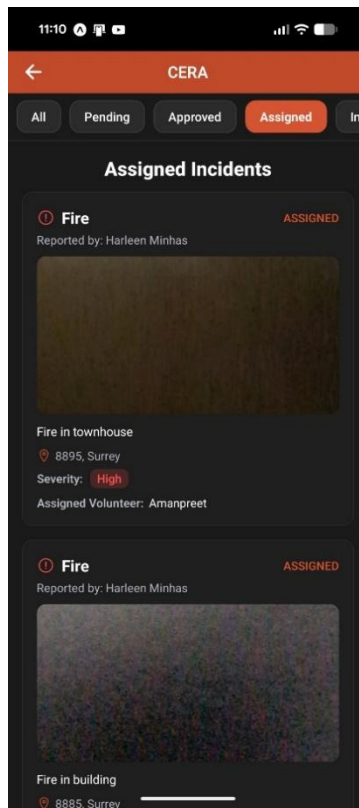
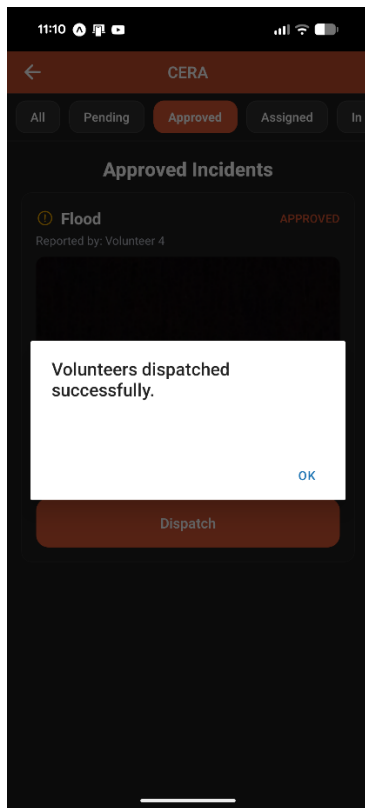
The coordinator dashboard includes UI elements such as the incident filter bar, incident cards with approval and dispatch buttons, and the volunteer selection modal used for assigning responders to an incident.

The dispatch modal displays the list of available volunteers and highlights selected users, allowing coordinators to quickly finalize assignments before updating the incident status.





Community Emergency Response App



Conclusion

The Coordinator Incident Management and Dispatch feature provides a centralized and efficient workflow for handling emergencies in CERA. By combining incident review, approval, volunteer assignment, and status tracking into a single interface, the module ensures that coordinators can manage emergencies with clarity and control. This feature is essential to the overall system, bridging the gap between emergency reports and real-world action by volunteers.

4.3 Feature C — Volunteer: Task Acceptance & Status Updates

Introduction

The Volunteer Task Management feature enables volunteers to respond to emergency assignments quickly and efficiently. Since volunteers form the operational core of the CERA response system, this feature ensures they receive timely notifications, understand the task details, and can communicate their availability in real time. The feature is designed to be simple, direct, and action-focused, allowing volunteers to either accept or decline assignments and update their progress as they respond to an incident.

Feature Description

When a coordinator dispatches volunteers for an incident, each selected volunteer receives a push notification prompting them to review the task. The volunteer can then open the assigned incident from their dashboard to view all relevant details, including the incident type, description, location, severity, and any supporting photo evidence.

Volunteers are given two primary actions:

- **Accept Task** — informs the system that the volunteer is taking responsibility for the incident.
- **Decline Task** — notifies the coordinator that the volunteer is unable or unavailable to respond.

Once a volunteer accepts a task, the incident status updates accordingly, and the volunteer becomes part of the active response team. If the volunteer declines the task, the system returns the incident to the coordinator's "Approved" list, allowing them to dispatch a different volunteer without delays.

The feature ensures that coordinators always have real-time visibility into volunteer responses and that volunteers have a clear, guided workflow for managing their assignments.

Assumptions Made During Implementation

The design assumes that volunteers actively keep their availability status updated so they are only shown tasks when they are ready to respond. It also assumes that volunteers will check notifications promptly and that manual accept/decline actions are preferable to automated responses.

Additionally, the system was developed with the expectation that volunteers may be handling one task at a time, helping maintain clarity and reduce overlap in responsibilities.

Changes Made During Development

A few adjustments were made during development to refine the volunteer experience and improve coordinator visibility:

- If a volunteer declines the assignment, the incident automatically returns to the Approved section for the coordinator, ensuring continuous workflow.
- The incident screen layout was improved so volunteers can clearly view the incident description, severity level, and photo evidence before making a decision.

Technical Implementation Details

This feature uses React Native components to render incident details and provide simple interaction buttons for accepting or declining tasks. When the volunteer accepts or declines an assignment, the app sends an API request to the backend:

- POST /incidents/:id/accept
- POST /incidents/:id/decline

The backend then updates the incident's volunteer assignment record and adjusts the incident status accordingly. MongoDB stores the updated logs, including timestamps and volunteer responses, ensuring coordinators receive an accurate timeline of actions.

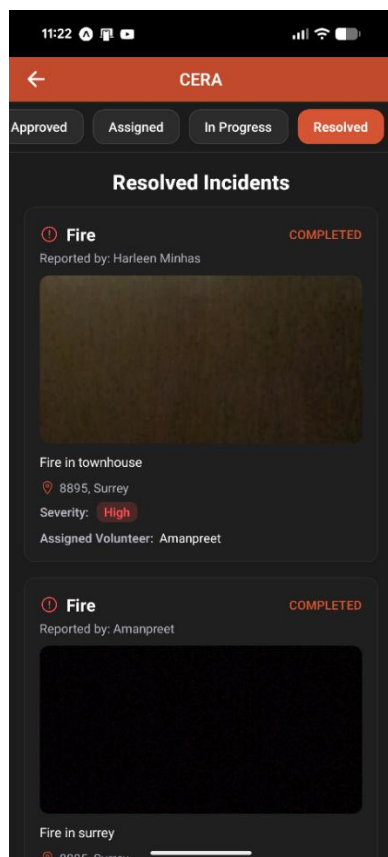
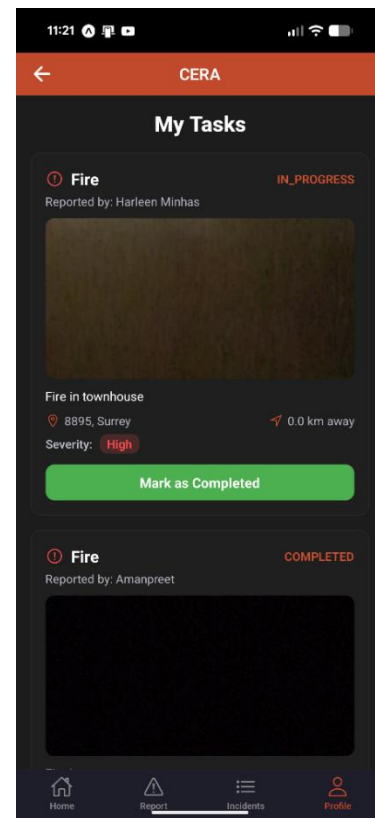
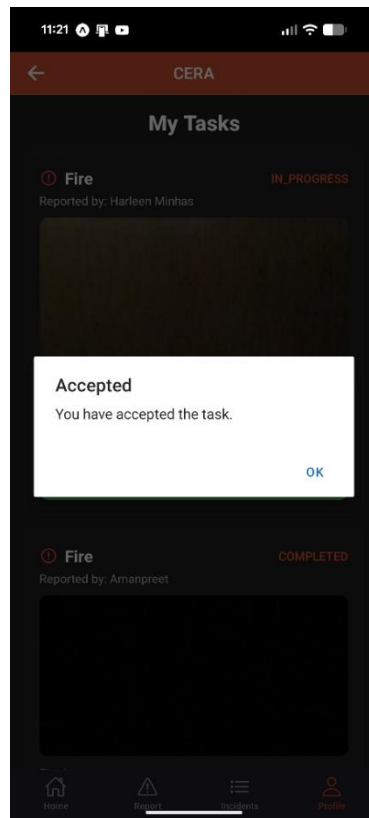
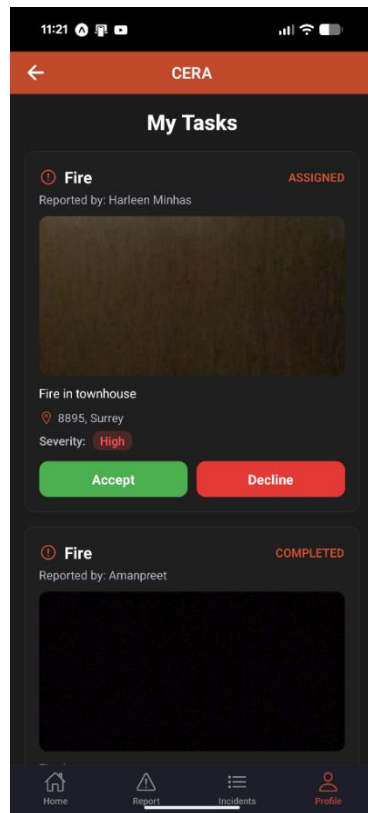
The volunteer dashboard fetches assigned incidents through the API, allowing volunteers to track their active tasks. If accepted, the volunteer's status and the incident's progress are updated instantly, allowing coordinators to monitor their activity in real time.

Screenshots

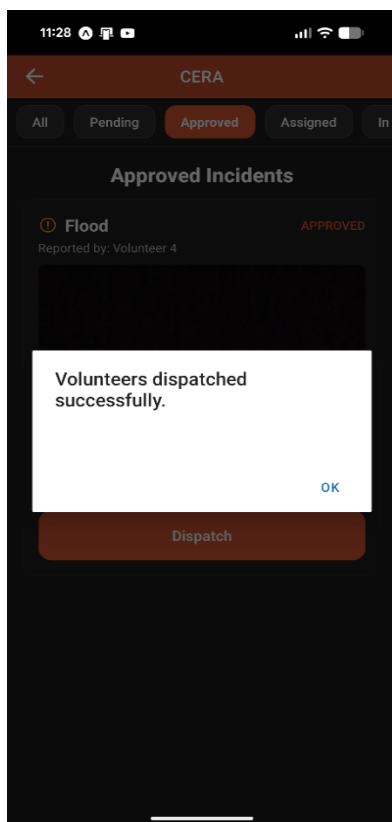
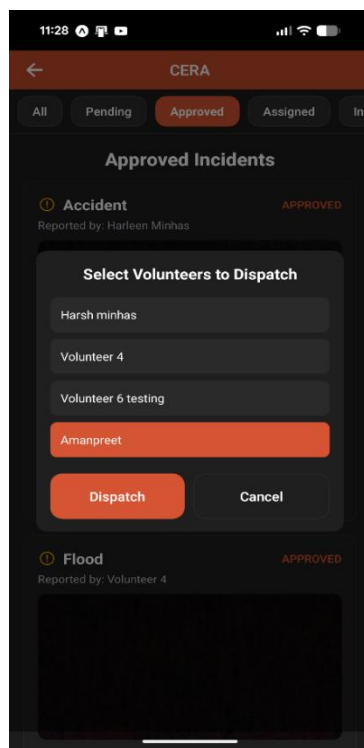
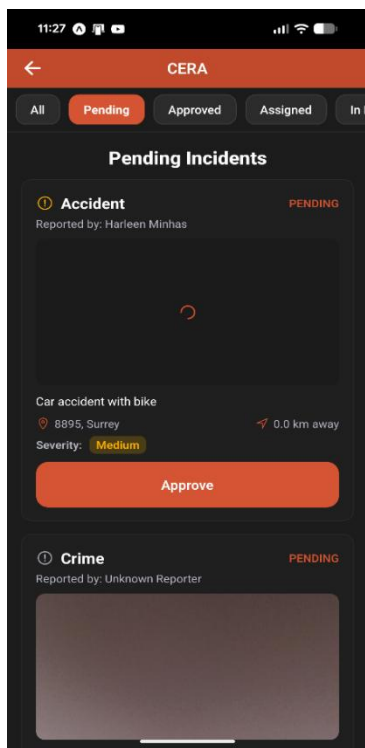
Screens show the volunteer's incident card with action buttons for accepting or declining the task. Once accepted, the interface displays updated task information and disables further modifications, ensuring clarity. If declined, a confirmation message appears and the task is removed from the volunteer's active list.

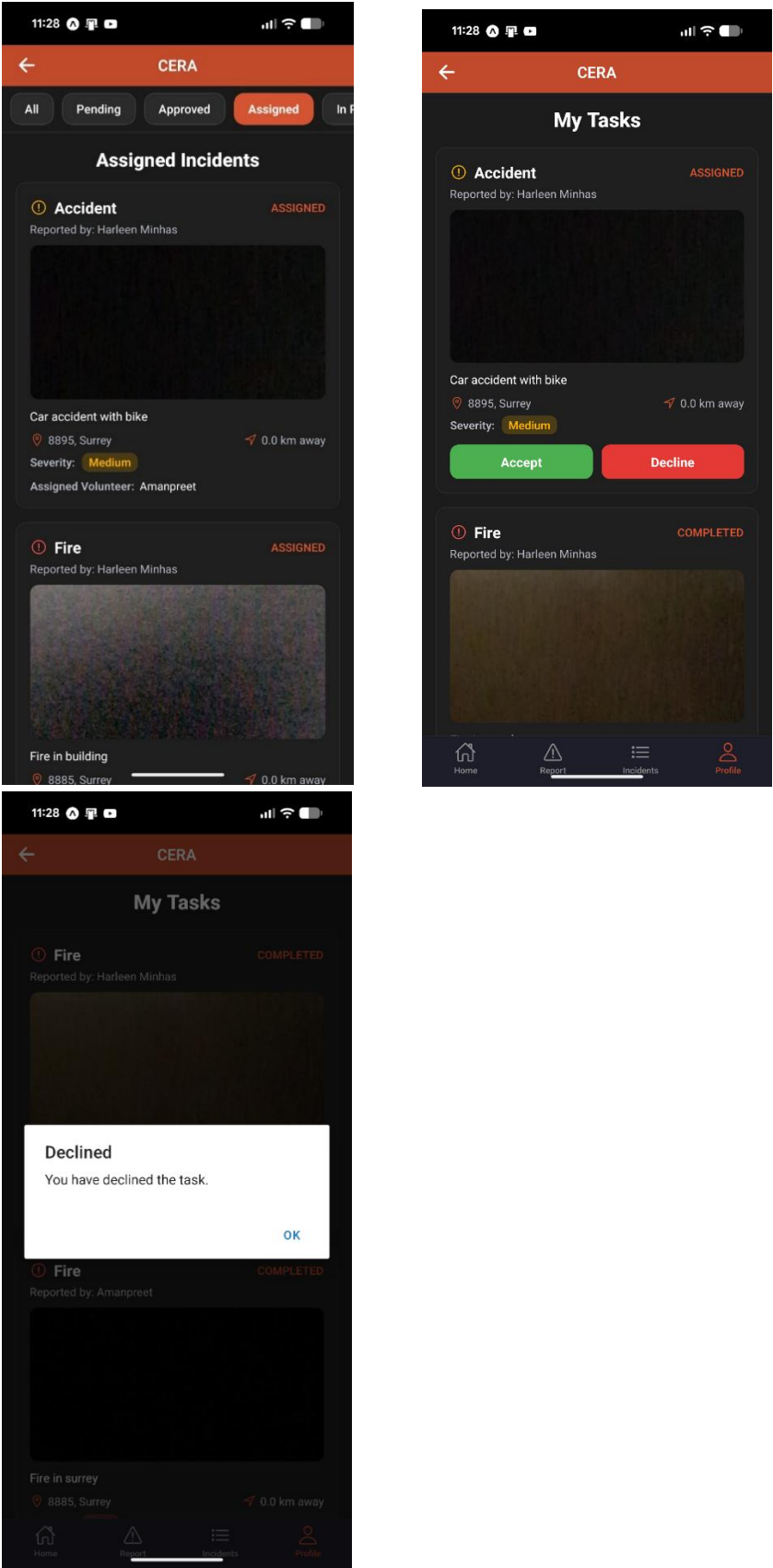
If Accept:

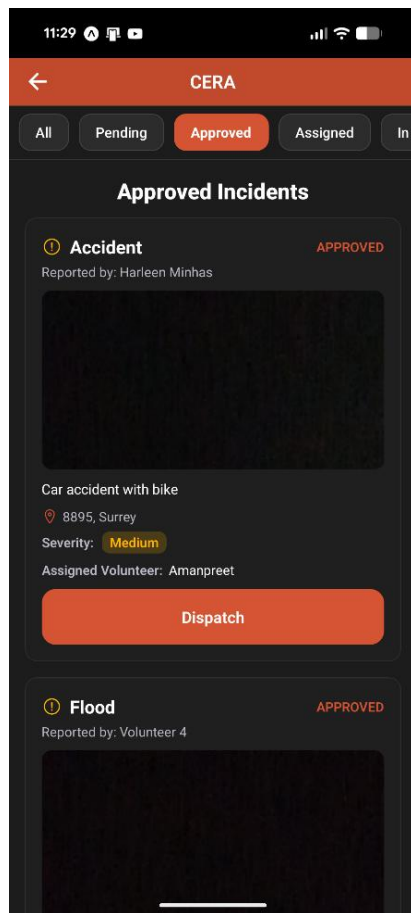
Community Emergency Response App



If task decline:







Conclusion

The Volunteer Task Acceptance and Status Update feature ensures that volunteers can respond quickly and confidently to assigned emergencies. By combining clear task information, responsive UI actions, and real-time backend updates, this module strengthens the entire emergency response workflow. It enables volunteers to participate effectively while providing coordinators with the visibility needed to make informed decisions.

4.4 Feature D — Coordinator: Incident Tracking

Introduction

The Incident Tracking feature provides coordinators with a centralized and real-time view of all emergencies reported within the system. This screen serves as the monitoring hub, where coordinators can review incident details, follow the progression of each case, see volunteer assignments, and navigate to detailed incident pages. Since coordinators are responsible for overseeing the full emergency workflow, the tracking feature ensures they always have visibility

into what is happening across the community at any moment. It acts as the final oversight layer after reporting, approval, and dispatching.

Feature Description

This feature displays a list of all incidents stored in the system, regardless of their status—pending, approved, assigned, in progress, resolved, or completed. Each incident is presented as a card showing key information such as the incident type, custom type (if provided), current status, timestamp of when it was reported, and the number of volunteers assigned.

Coordinators can tap on any incident to navigate to a detailed incident view, where they can see the complete report, attached photos, location coordinates, assigned volunteers, logs, and status history. This makes it easier to track how volunteers are handling the situation and whether any follow-up action is needed. The tracking screen updates every time the coordinator refreshes it, ensuring that the list always reflects the latest state of the system.

Assumptions Made During Implementation

The feature was implemented with the assumption that coordinators would need quick access to all incidents without any pre-filtering. It assumes that a timeline-based view helps coordinators understand the flow of reports and the urgency of different cases. It also assumes that coordinators have stable internet access, since the feature relies on fetching real-time data from the backend.

Another assumption is that coordinators are familiar with reviewing multiple cases simultaneously and can navigate between screens easily. This design supports rapid monitoring by allowing them to open detailed pages as needed, rather than cluttering the list with too much information.

Changes Made During Development

During testing, several adjustments were made to improve reliability and clarity:

The API call was extended to fetch all incidents, including resolved and completed ones, so coordinators could review entire histories rather than just active cases.

A more structured card layout was introduced to ensure that status labels, timestamps, and volunteer counts were easy to scan.

Technical Implementation Details

The tracking feature was implemented using React Native and integrates closely with the backend API. When the screen loads, it retrieves the JWT token stored in AsyncStorage and

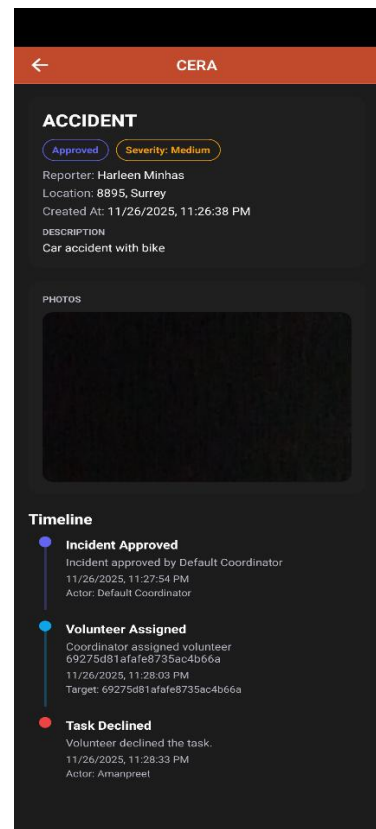
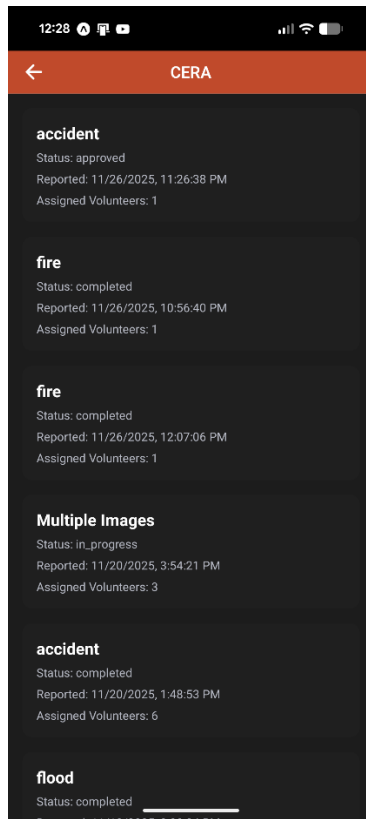
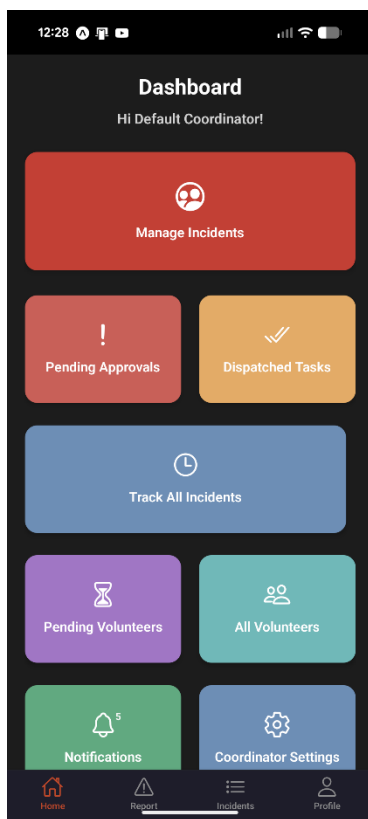
attaches it to the Authorization header for secure requests. The system then calls the /incidents endpoint, which returns the full list of incident records stored in MongoDB.

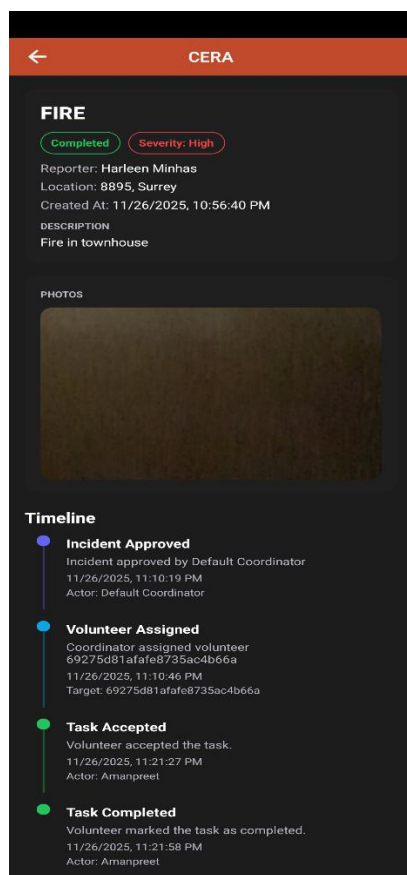
Each incident object contains the type, custom type, reporter information, timestamps, assigned volunteers, and current status. The frontend displays this information in simplified cards, and the entire list is wrapped in a ScrollView for smooth navigation.

The React Native screen updates dynamically by calling the load() function, which fetches fresh data each time the user returns to the page or manually refreshes it. This creates a real-time-like tracking experience without requiring background subscriptions.

Screenshots Description

The Incident Tracking screen displays a vertical list of incidents presented in clean, card-style components. Each card includes the incident name, custom type if available, the current status, the time it was reported, and the number of assigned volunteers. Tapping a card opens the detailed view, where coordinators can see the full report, photo, and logs.





Conclusion

The Incident Tracking feature plays a critical role in allowing coordinators to maintain oversight of the entire emergency workflow. By offering a centralized view of all incidents, this feature supports quick decision-making, transparent progress monitoring, and efficient coordination with volunteers. With real-time updates, clear navigation, and reliable data retrieval, the tracking module enhances the platform's overall effectiveness and ensures that emergencies can be reviewed and managed with confidence and clarity.

REFLECTIONS

The development of the Community Emergency Response App (CERA) has been a highly meaningful and transformative learning experience. Working on this project allowed us to combine technical skills, research findings, and real-world problem-solving approaches into a complete, functional system. Throughout the process, we gained a deeper understanding of both the technical and human aspects of designing an emergency management solution.

One major learning outcome was understanding how important simplicity and clarity are in high-stress contexts. While building features such as incident reporting, volunteer dispatch, and tracking, we realized that even small interface decisions—such as how many steps a form should have or where a button should be placed—could significantly impact usability. Much of our

refinement came from testing, observing confusion points, and iterating the design to make interactions more intuitive.

Another important reflection was the complexity of coordinating multiple user roles (resident, volunteer, coordinator) within a single system. Each role behaves differently and requires separate workflows, permissions, and API interactions. Designing these flows helped us better understand the importance of role-based access, proper data separation, and secure API handling. This experience strengthened our ability to architect multi-layered applications rather than isolated features.

From a backend perspective, handling geolocation, filtering, volunteer availability, and incident statuses taught us how to manage dynamic, real-time data. Implementing task acceptance, decline, and completion flows showed us how small logic details can impact the overall system behavior. Debugging these flows also helped improve our understanding of asynchronous operations, JWT authentication, and MongoDB queries.

We also encountered several challenges that shaped the final outcome. Device location accuracy varied across environments, image uploads needed additional optimization, and push notifications required careful handling of tokens and platform differences. While some features proposed initially—such as offline support—could not be completed due to time constraints, the experience helped us learn how to prioritize essential functionality while keeping long-term scalability in mind.

Working as a team also contributed significantly to our learning. Coordinating tasks, merging code on GitHub, resolving conflicts, and maintaining consistent documentation taught us the importance of collaboration in software development. Dividing responsibilities allowed each member to focus deeply on specific parts while still maintaining a shared understanding of the entire system.

Overall, this project strengthened our technical competence, improved our problem-solving approach, and helped us appreciate the responsibility that comes with building tools intended for emergency situations. If we were to continue developing CERA, we would focus on optimizing performance, improving real-time tracking, integrating offline functionality, and conducting user testing with larger groups to further refine the experience. The project has given us confidence in our ability to design meaningful, user-centered applications and has prepared us for more complex development challenges in the future.

CONCLUDING REMARK

The development of the Community Emergency Response App (CERA) demonstrated how mobile technology can support faster and more organized emergency communication within a community. Through the implementation of core features—incident reporting, volunteer task management, incident tracking, and coordinator oversight—the system successfully creates a structured workflow that replaces scattered communication methods.

Although the project faced limitations such as time constraints and the exclusion of offline mode, it still achieved a functional prototype that reflects the planned design and research objectives. The experience provided valuable insights into real-time communication, mobile UI design, backend integration, and the challenges of coordinating multiple user roles within a single application.

Overall, CERA lays the foundation for a scalable emergency response tool that can be further expanded and integrated with official emergency services. The project highlights the potential for technology-driven, community-based solutions to improve safety, coordination, and response readiness.

Appendix A — Installation Guide

This guide explains how to set up and run the CERA system on a new machine. It includes instructions for cloning the repository, configuring the backend API, installing the mobile application, and running both parts together.

A.1 Git Setup & Project Retrieval

Before running the project, the first step is to download the code from GitHub.

A.1.1 Clone the Repository

1. Open a terminal or command prompt.
2. Navigate to your preferred folder.
3. Run:

```
git clone https://github.com/dappysandhu/F2025\_4495\_050\_Hka855.git
```

4. Move into the project folder:

```
cd F2025_4495_050_HKa855/Implementation
```

A.2 Backend Setup (Node.js + Express + MongoDB)

The backend provides authentication, incident reporting, volunteer dispatch, notifications, and all API endpoints.

A.2.1 Requirements

- Node.js
- npm or yarn
- Internet connection

Note:

The project uses MongoDB Atlas, so testers do not need to install MongoDB locally. The connection string is preconfigured in the .env file provided by the developers.

A.2.2 Install Dependencies

Navigate to the backend folder:

```
cd Implementation/backend
```

```
npm install
```

A.2.3 Environment Configuration

A .env file must exist with:

```
MONGO_URI=mongodb+srv://majorproject:cera@cera.bqujud.mongodb.net/test?retryWrites=true&w=majority&appName=CERA
```

```
JWT_SECRET=supersecret
```

```
PORT=5000
```

```
CLOUDINARY_URL=cloudinary://817326472365777:IyKUAqEt6gT267gfDebjmdSXLc4@dbaykrqox
```

```
CLOUDINARY_FOLDER=cera/incidents
```

A.2.4 Run the Backend API

```
npm start
```

If successful, the console will display:

```
Connected to MongoDB successfully
```

Server running on <http://localhost:5000>

A.3 Mobile App Installation (React Native + Expo)

The mobile application supports residents, volunteers, and coordinators.

A.3.1 Requirements

- Node.js installed
- Expo CLI installed
- Android device or emulator

A.3.2 Install the Expo App (Mobile)

Install Expo Go from Play Store (for Android testing).

A.3.3 Install Mobile Dependencies

Navigate to the mobile folder:

```
cd Implementation/mobile/cera-mobile  
npm install
```

A.3.4 Configure API Base URL

Inside:

```
cera-mobile/constants/config.ts
```

Set:

```
export const API_BASE_URL = "http://<your-local-ip>:5000/api";
```

Both the laptop and the mobile device must be connected to the same Wi-Fi network.

A.3.5 Start the Mobile App

```
npx expo start
```

Scan the QR code using the Expo Go app.

A.4 Common Setup Issues & Fixes

Issue: Backend not reachable

- Make sure both devices are on the same network.

Issue: API Base URL incorrect

- Use local IP, not localhost.

Issue: Metro Bundler stuck

Run:

```
npm start --clear
```

Appendix B — User Guide



CERA-UserGuide.doc

x

Appendix C — Dataset and API Used

This appendix provides an overview of the datasets, backend APIs, and core endpoints used in the Community Emergency Response App (CERA). Since the system generates its own data dynamically through user interactions, no external datasets were used. The backend API, built using Node.js and Express, enables all communication between the mobile application and the server, including authentication, incident reporting, volunteer dispatch, and location tracking.

C.1 Dataset Used

CERA does **not** use third-party datasets.

Instead, all data is generated internally through user activity. The main collections stored in **MongoDB Atlas** include:

- **Users** (Residents, Volunteers, Coordinators)
- **Incident Reports** (description, severity, photos, coordinates)
- **Volunteer Assignments** (accepted, declined, completed statuses)
- **Incident Logs** (timestamped actions for tracking history)
- **Emergency Contacts** (saved by residents)

- **Push Tokens** (for sending notifications)

This internal dataset supports the full workflow of reporting, dispatching, and tracking emergencies.

C.2 API Overview

The backend exposes a RESTful API with JWT-based authentication.

All endpoints are secured and require valid tokens except registration and login.

APIs are grouped into the following categories:

1. Authentication API
2. User & Volunteer Management API
3. Incident Reporting API
4. Incident Location & Filtering API
5. Volunteer Dispatch & Response API
6. Notification API

Below is the complete list of endpoints used in the project.

C.3 API Endpoints

1. Authentication API

POST /auth/register

Registers a new user (Resident/Volunteer/Coordinator). Validates email, phone, role, and password. Uncertified volunteers require approval.

POST /auth/login

Returns token & user data after validating credentials.

GET /auth/me

Returns the profile of the logged-in user (JWT required).

2. User & Volunteer Management API

GET /users

Fetches users. Supports filters:
role, approved, certified, available.

GET /users/pending

Coordinator views volunteers awaiting approval.

POST /users/:id/approve

Coordinator approves volunteer (certified + approved).

POST /users/:id/decline

Coordinator declines volunteer request and deletes the user.

PATCH /users/me

Updates user profile (username, phone, email, skills).

PATCH /users/me/location

Updates user's live GPS coordinates (GeoJSON).

POST /users/me/push-token

Stores Expo push notification token.

POST /users/me/emergency-contacts

Adds an emergency contact for residents.

DELETE /users/me/emergency-contacts/:index

Deletes a saved contact.

GET /users/nearby

Returns nearby available volunteers based on coordinates.

3. Incident Reporting API

POST /incidents

Creates a new incident with:

- Type & custom type
- Description
- Severity

- Affected count
- Location (GeoJSON)
- Images uploaded to Cloudinary

Automatically notifies coordinators.

GET /incidents

Returns all incidents (optional filters: status, type).

GET /incidents/my

Returns incidents reported by the logged-in resident.

GET /incidents/:id

Returns detailed incident view including logs & assigned volunteers.

4. Incident Location & Filtering API

GET /incidents/nearby

For volunteers: fetches incidents within radius, filtered by: type, severity, status, unassigned only.

GET /incidents/assigned/me

Returns incidents assigned to the volunteer.

GET /incidents/volunteer/completed

Returns incidents completed by logged-in volunteer.

5. Volunteer Dispatch & Response API

POST /incidents/:id/approve

Coordinator approves an incident.

POST /incidents/:id/dispatch

Coordinator assigns volunteers to incident.

POST /incidents/:id/accept

Volunteer accepts the assigned task.

POST /incidents/:id/decline

Volunteer declines the task; incident returns to “Approved”.

POST /incidents/:id/complete

Volunteer marks task as completed; status becomes “completed”.

6. Notification API

The notification system uses internal functions (notifyUser) and Expo Push Notification Service. Notifications are triggered automatically when:

- A volunteer registers → coordinators are notified
- A new incident is created → coordinators receive alert
- Coordinator dispatches task → volunteer receives assignment
- Volunteer accepts/declines → coordinators notified
- Volunteer completes task → coordinators notified
- Resident requests help → nearby volunteer notified

There is no public notification endpoint; it is event-driven.

C.4 Conclusion

The API provides a complete, secure, and scalable communication layer for the CERA mobile application. By separating authentication, user management, incident workflows, geolocation queries, and volunteer coordination, the system maintains clarity and modularity. This structured API design supports real-time emergency management and ensures that every interaction—from reporting to dispatch—occurs smoothly and efficiently.

CONCLUSION

The Community Emergency Response App (CERA) successfully demonstrates how a mobile-first system can streamline communication and coordination during community emergencies. Through the implementation of core features—incident reporting, volunteer dispatch, real-time tracking, and role-based workflows—the project delivers a functional prototype that addresses the major gaps identified in the research phase.

This project not only achieved its technical objectives but also provided valuable learning in system design, multi-role architecture, backend integration, geolocation handling, and mobile UI development. While certain features such as offline mode and advanced automation remain future enhancements, the completed system provides a solid foundation for a scalable, community-driven emergency response solution.

CERA emphasizes the importance of timely information flow, structured decision-making, and user-centered design during high-stress situations. With further refinement, testing, and expansion, the platform has the potential to support real-world adoption by local communities, volunteer groups, and emergency coordinators. Overall, the project demonstrates the strong potential of technology-driven solutions to enhance community safety, preparedness, and response effectiveness.